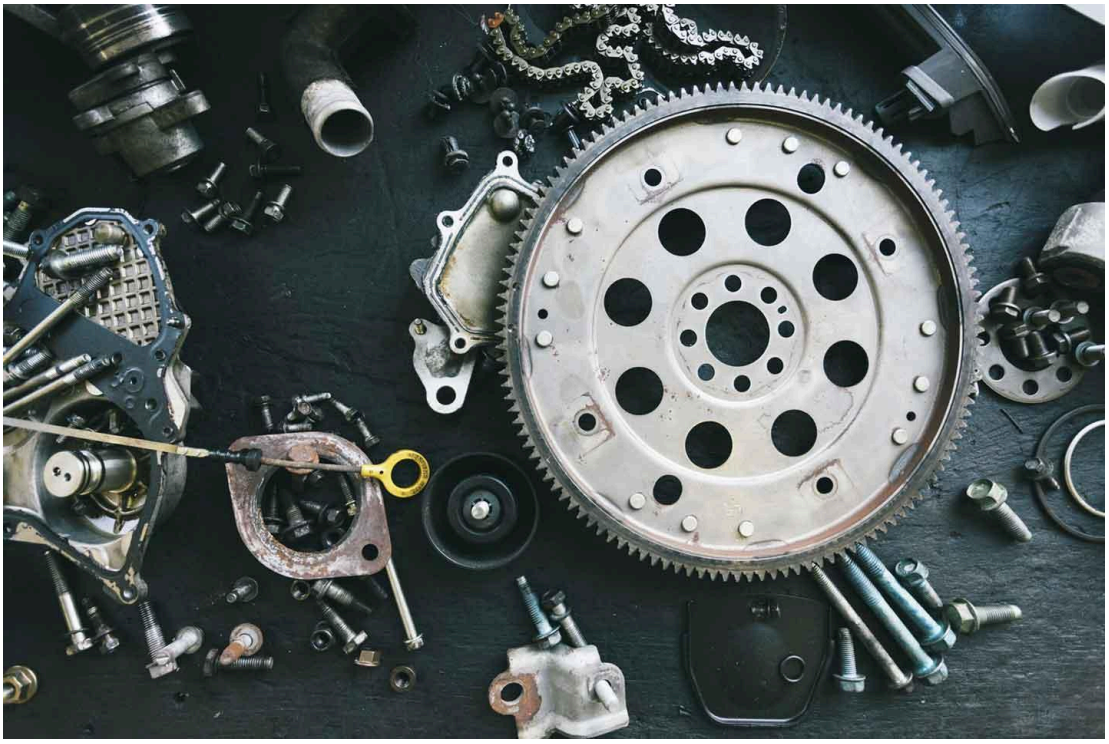
Tema 4

Capa de Servicio y Controladores REST

Digitech
FP OFICIAL DIGITAL & TECH



Francisco Belda

ÍNDICE

Teoría. Arquitectura en Capas **3**
Paso a Paso: Creación de la Capa de Servicio **4**
Creación del paquete service 4
Implementación de EstudianteService 4
Explicación 5
Práctica: Creación del Controlador REST de Estudiantes **6**
Creación del EstudianteController 6
Desglose técnico de anotaciones 8
Ejercicios **9**
Conceptos Clave Recordados 9
Ejercicios 9

Teoría. Arquitectura en Capas

Para mantener un código limpio, mantenible y escalable, las responsabilidades de la aplicación se dividen en tres capas principales:

- **Capa de Presentación / Controladores (controller)**: Recibe las peticiones HTTP del cliente, valida los datos de entrada superficialmente y devuelve una respuesta HTTP (generalmente convertida a JSON).
- **Capa de Negocio / Servicios (service)**: Contiene la lógica de negocio de la aplicación (cálculos, validaciones complejas, transacciones). No sabe nada de HTTP ni de navegadores web.
- **Capa de Persistencia / Repositorios (repository)**: Se comunica directamente con la base de datos utilizando Spring Data JPA.

Paso a Paso: Creación de la Capa de Servicio

Creación del paquete service

Crea un nuevo paquete llamado **service** dentro de tu estructura raíz (ej. **com.educacion.demospring.service**).

Implementación de EstudianteService

Crea una clase llamada **EstudianteService.java** dentro del paquete **service**:

```
package com.educacion.demospring.service;


import com.educacion.demospring.model.Estudiante;

import
com.educacion.demospring.repository.EstudianteRepository;

import org.springframework.stereotype.Service;

import java.util.List;

import java.util.Optional;


@Service

public class EstudianteService {


   private final EstudianteRepository estudianteRepository;


   // Inyección de dependencias a través del constructor

            public     EstudianteService(EstudianteRepository
estudianteRepository) {

       this.estudianteRepository = estudianteRepository;
```

```
    }
    // Obtener todos los estudiantes
    public List<Estudiante> listarTodos() {
        return estudianteRepository.findAll();
    }
    // Buscar estudiante por ID
    public Optional<Estudiante> buscarPorId(Long id) {
        return estudianteRepository.findById(id);
    }
    // Guardar o actualizar un estudiante
    public Estudiante guardarEstudiante(Estudiante estudiante)
{
        return estudianteRepository.save(estudiante);
    }
    // Eliminar un estudiante por ID
    public void eliminarEstudiante(Long id) {
        estudianteRepository.deleteById(id);
    }
// Buscar si existe el Correo
public boolean existsByCorreo(String correo) {
    return estudianteRepository.existsByCorreo(correo);
}
}
```

Explicación

La anotación @Service le indica a Spring que esta clase es un componente de negocio gestionado por el contenedor IoC. Gracias a la inyección por constructor, Spring le pasa automáticamente la instancia del repositorio.

Práctica: Creación del Controlador REST de Estudiantes

Creación del EstudianteController

Crea una clase llamada **EstudianteController.java** dentro del paquete **controller**:

```
package com.educacion.demospring.controller;

import com.educacion.demospring.model.Estudiante;

import com.educacion.demospring.service.EstudianteService;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;

import java.util.List;


@RestController

@RequestMapping("/api/v1/estudiantes")

public class EstudianteController {


   private final EstudianteService estudianteService;


            public    EstudianteController(EstudianteService
estudianteService){

       this.estudianteService = estudianteService;

   }


   // GET: Listar todos los estudiantes

   @GetMapping

   public List<Estudiante> obtenerTodos() {
```

```
        return estudianteService.listarTodos();
    }

    // GET: Buscar estudiante por ID
    @GetMapping("/{id}")
     public ResponseEntity<?> obtenerPorId(@PathVariable Long
id) {
        return estudianteService.buscarPorId(id)
                .<ResponseEntity<?>>map(ResponseEntity::ok)
                                            .orElseGet(()   ->
ResponseEntity.status(HttpStatus.NOT_FOUND)

.body(Collections.singletonMap("mensaje",  "El  estudiante  no
existe")));
    }

// POST: Crear un nuevo estudiante
@PostMapping
public     ResponseEntity<?>     crearEstudiante(@RequestBody
Estudiante estudiante) {
    // Validamos si el email ya está registrado
                                                          if
(estudianteService.existsByCorreo(estudiante.getCorreo())) {
        return ResponseEntity
                .status(HttpStatus.CONFLICT)
                 .body(Collections.singletonMap("mensaje", "El
correo ya está registrado por otro estudiante"));
    }
    // Guardamos el estudiante si no existe
                    Estudiante       nuevoEstudiante       =
estudianteService.guardarEstudiante(estudiante);
```

```
                                                    return
ResponseEntity.status(HttpStatus.CREATED).body(nuevoEstudiant
e);
}

    // PUT: Actualización completa
    @PutMapping("/{id}")
     public ResponseEntity<?> actualizarCompleto(@PathVariable
Long id, @RequestBody Estudiante estudianteDatos) {
        return estudianteService.buscarPorId(id)
                .<ResponseEntity<?>>map(estudianteExistente ->
{
estudianteExistente.setNombre(estudianteDatos.getNombre());
estudianteExistente.setEdad(estudianteDatos.getEdad());
                                        Estudiante actualizado =
estudianteService.guardarEstudiante(estudianteExistente);
                    return ResponseEntity.ok(actualizado);
                })
                                                .orElseGet(() ->
ResponseEntity.status(HttpStatus.NOT_FOUND)
.body(Collections.singletonMap("mensaje", "El estudiante no
existe")));
    }

    // PATCH: Actualización parcial
    @PatchMapping("/{id}")
     public ResponseEntity<?> actualizarParcial(@PathVariable
Long id, @RequestBody Map<String, Object> campos) {
```

```
        return estudianteService.buscarPorId(id)
                .<ResponseEntity<?>>map(estudianteExistente ->
{
                    campos.forEach((campo, valor) -> {
                        switch (campo) {
                            case "nombre":

estudianteExistente.setNombre((String) valor);
                                break;
                            case "correo":

estudianteExistente.setCorreo((String) valor);
                                break;
                            case "edad":
                                if (valor instanceof Number) {

estudianteExistente.setEdad(((Number) valor).intValue());
                                }
                                break;
                        }
                    });

                                    Estudiante actualizado =
estudianteService.guardarEstudiante(estudianteExistente);
                    return ResponseEntity.ok(actualizado);
                })
                                            .orElseGet(() ->
ResponseEntity.status(HttpStatus.NOT_FOUND)

.body(Collections.singletonMap("mensaje", "El estudiante no
existe")));
    }
```

```
    // DELETE: Eliminar un estudiante
    @DeleteMapping("/{id}")
    public ResponseEntity<?> eliminarEstudiante(@PathVariable
Long id) {
        if (estudianteService.buscarPorId(id).isPresent()) {
            estudianteService.eliminarEstudiante(id);
return  ResponseEntity.ok(Collections.singletonMap("mensaje",
"Usuario borrado"));

        }
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body(Collections.singletonMap("mensaje", "El
estudiante no existe"));
    }
}
```

Desglose técnico de anotaciones

- **@RequestMapping("/api/estudiantes")**: Prefijo de ruta base para todos los endpoints de este controlador.
- **@PathVariable**: Extrae variables directamente de la URL (ej. /api/estudiantes/1).
- **@RequestBody**: Convierte automáticamente el JSON recibido en el cuerpo de la petición HTTP POST a un objeto Java (Estudiante).
- **ResponseEntity**: Permite controlar de forma precisa los códigos de estado HTTP devueltos al cliente (200 OK, 201 Created, 404 Not Found, 204 No Content).

Ejercicios

Conceptos Clave Recordados

La arquitectura en capas desacopla la lógica de negocio (Servicios) del acceso a datos (Repositorios) y de la interfaz web (Controladores).

@Service registra la clase de lógica de negocio en el contenedor IoC de Spring.

Los métodos POST utilizan @RequestBody para deserializar JSON a objetos Java, mientras que las respuestas con códigos de estado HTTP precisos se gestionan mediante ResponseEntity.

Ejercicios

1. **Crear el Servicio de Profesores**: Implementa la clase **ProfesorService** inyectando el **ProfesorRepository** y añadiendo **métodos para listar todos los profesores, buscarlos por ID, actualizar sus datos y guardarlos**.
2. **Crear el Controlador de Profesores**: Desarrolla el **ProfesorController** expuesto en la ruta **/api/profesores** que permita realizar peticiones **GET (listar todos y buscar por ID) y POST (crear profesor)** haciendo uso del servicio creado en el ejercicio anterior. También tendrá que ofrecer **PATCH** y **PUT (actualizar parcial y completo)**. Además ofrecerá el **DELETE (borrar**).